Machine Learning

Solutions to Tutorial 2: Learning by Combining Models

Detailed, step-by-step solutions in the notation of Lecture 5 and of Chapter 5 of the textbook.

How to use these solutions

The setup of each problem (the scenario, the data, the tables) is restated in a blue box. Each sub-question then appears in its own highlighted bar, word for word, *directly above its answer*, so you never have to scroll back to see what is being asked. Each solution opens with a yellow box saying which concept the problem tests. References like [Chapter 5, §5.2] point to the section of *Chapter 5: Learning by Combining Models* where the concept is explained and proved. Numbers are worked to three or four significant digits so you can check your own calculations line by line. The sheet's instruction applies throughout: explain your reasoning and state your assumptions; where the question leaves a convention open (a tie at exactly 0.5, a gain of exactly zero) the assumption is stated explicitly.

The thread so far

The central question of the tutorial: *how can several models be combined so that their different errors improve prediction, while controlling complexity and evaluating the resulting ensemble honestly?* The nine problems walk through the chapter in order: combination rules (1), decorrelating trees (2), reading importance (3), fitting residuals with shrinkage (4), Newton leaf values (5), XGBoost's split gain (6), model selection from validation results (7), leakage in ensemble experiments (8), and deployment under shift (9).

Contents

1 Problem 1: Hard voting, soft voting, and confidence **2**

2 Problem 2: Why can weaker trees form a stronger forest? **3**

3 Problem 3: Interpreting feature importance **5**

4 Problem 4: Fitting residuals with shrinkage **6**

5 Problem 5: From pseudo-residuals to Newton leaf values **8**

6 Problem 6: A split must pay for its extra leaf **10**

7 Problem 7: Choosing a model from validation results **11**

8 Problem 8: Avoiding leakage in an ensemble experiment **13**

9 Problem 9: Deployment when the data may change **15**

1 Problem 1: Hard voting, soft voting, and confidence

Problem 1

Three classifiers predict the probability of an attack for four flows:

Flow	p_A	p_B	p_C	y
1	0.49	0.49	0.95	1
2	0.80	0.55	0.05	1
3	0.40	0.40	0.90	0
4	0.10	0.70	0.70	1

Each individual classifier uses threshold 0.5.

What this tests

Hard voting counts class votes; soft voting averages posteriors and thresholds the average [Chapter 5, §5.2]. The two rules disagree exactly when the *margins* matter: soft voting uses how far each probability is from 0.5, hard voting discards it. Soft voting therefore depends on the probabilities being calibrated and comparable; an overconfident model dominates it.

(a) Find the hard-voting and soft-voting prediction for each flow. Whenever they disagree, explain what information one rule uses that the other discards.

First convert each probability to an individual vote with the threshold 0.5 (a probability ≥ 0.5 votes attack, 1), then count; for soft voting average the three probabilities and apply the same threshold to the average.

Flow	A	B	C	hard vote	$\bar{p} = (p_A + p_B + p_C)/3$	soft vote	y
1	0	0	1	**0** (2–1)	$1.93/3 = 0.643$	**1**	1
2	1	1	0	**1** (2–1)	$1.40/3 = 0.467$	**0**	1
3	0	0	1	**0** (2–1)	$1.70/3 = 0.567$	**1**	0
4	0	1	1	**1** (2–1)	$1.50/3 = 0.500$	**1** (tie)	1

Flow 4 is a tie for soft voting: the average is exactly 0.500. With the convention "$\geq 0.5 \Rightarrow$ attack" (the natural one for a detector, where a miss costs more than a false alarm) soft voting predicts 1 and agrees with hard voting; with a strict "> 0.5" convention it would predict 0. State the assumption; the rest of the answer uses "$\geq$".

Scorecard. Hard voting: $0, 1, 0, 1$, correct on flows 2, 3, 4 (three of four). Soft voting: $1, 0, 1, 1$, correct on flows 1 and 4 (two of four).

Where they disagree and why.

- *Flow 1* (hard 0, soft 1). A and B are coin flips, 0.49 each, barely on the benign side; C is almost certain, 0.95. Soft voting uses these *margins*: two near-zero margins against one huge margin, so C wins the average. Hard voting sees only two benign votes against one attack vote and discards how weak the two benign votes were. Here soft voting is right.
- *Flow 2* (hard 1, soft 0). A and B lean attack (0.80, 0.55); C is confidently benign (0.05). Soft voting lets C's large margin (0.45 below the threshold) outweigh A's (0.30 above) and B's (0.05 above): $0.80 + 0.55 + 0.05 = 1.40 < 1.5$. Hard voting discards C's confidence and counts heads. Here hard voting is right.
- *Flow 3* (hard 0, soft 1). Same mechanism as flow 1 with C at 0.90 pulling the average over 0.5, but this time C is wrong: the flow is benign. Hard voting is right.

In every disagreement the information at stake is the same: soft voting uses the distance of each probability from the threshold; hard voting throws it away and treats 0.49 and 0.05 identically. Whether that information helps depends on whether the confident model deserves its confidence.

(b) Suppose classifier C is often too confident. Explain how this affects hard and soft voting. Does this mean that hard voting is always better? Give a counterexample.

Effect on soft voting. An overconfident C outputs probabilities near 0 or 1 even when it is wrong. In an average of three, a term of 0.95 or 0.05 moves $\bar{p}$ by about ± 0.15 relative to a neutral 0.5, while two moderate models at 0.4–0.6 move it by at most ± 0.07 together. C therefore *dominates* the soft vote: flows 1, 2 and 3 were all decided by C. When C is right (flow 1) the ensemble is right; when C is wrong (flows 2, 3) it drags the ensemble with it. Soft voting assumes reasonably calibrated probabilities under the same class prior [Chapter 5, §5.2]; C violates that assumption, so its "confidence" is noise amplified by the averaging.

Effect on hard voting. C gets exactly one vote whatever its probability says. Its overconfidence is *capped*: it can be outvoted by A and B together, as on flows 2 and 3. Hard voting is robust to one miscalibrated member.

Is hard voting therefore always better? No. Hard voting is robust because it discards information, and sometimes that information is exactly what is needed.

- *Counterexample from the table.* Flow 1: A and B at 0.49 are saying "we have no idea", and C at 0.95 is right. Hard voting outputs benign on the strength of two coin flips and misses the attack. Soft voting detects it.
- *General counterexample.* Two models at 0.51 (wrong) and one well-calibrated model at 0.99 (right): hard voting is wrong 2–1; soft voting gives $(0.51 + 0.51 + 0.01)/3 = 0.343$, correct. Any time the majority is marginal and the minority is confidently and *correctly* so, hard voting loses.

The right response to an overconfident C is not to abandon soft voting but to fix the assumption it violates: *calibrate* C (e.g. on validation data), or give the models *weights* learned on validation data, which is exactly what stacking does [Chapter 5, §5.12]. With calibrated probabilities soft voting is the better rule, because it uses strictly more information; with miscalibrated ones hard voting is the safer rule, because it caps the damage.

Do not confuse

"Hard voting won 3–2 on this table" is not evidence that hard voting is better in general. Four flows is not a validation set, and the result is driven entirely by whether C happened to be right. The question is about the *mechanism*: what each rule uses and what it discards.

2 Problem 2: Why can weaker trees form a stronger forest?

Problem 2

A dataset has $d = 25$ features. Feature x_1 is highly predictive, so many bagged trees choose it at the root. A random forest offers only $m_{\text{try}} = 5$ randomly selected features at each split.

What this tests

Random forests add feature subsampling to bagging to *decorrelate* the trees [Chapter 5, §5.5]. The pay-off is governed by the variance formula $\text{Var} = \sigma^2(\rho + (1-\rho)/M)$ [Chapter 5, §5.2]: a small rise in each tree's error σ^2 is worth a large fall in the correlation ρ. Hyperparameters are chosen for the *forest*, by OOB or validation error [Chapter 5, §5.4, 5.5].

(a) Derive the probability that x_1 is available at one split and the probability that it is unavailable at the roots of all $M = 10$ trees.

At a node, $m_{\text{try}} = 5$ of the $d = 25$ features are drawn uniformly without replacement. By symmetry every feature has the same chance of being drawn, and 5 features are drawn, so

$$\Pr(x_1 \text{ available at one split}) = \frac{m_{\text{try}}}{d} = \frac{5}{25} = 0.2.$$

(Check by counting: subsets containing x_1 are $\binom{24}{4}$ out of $\binom{25}{5}$, and $\binom{24}{4}/\binom{25}{5} = 5/25$.) Hence x_1 is *unavailable* at one root with probability $1 - 0.2 = 0.8$. The candidate subsets at the ten roots are drawn independently, so

$$\Pr(x_1 \text{ unavailable at all 10 roots}) = 0.8^{10} \approx 0.107.$$

Equivalently x_1 appears at the root of at least one tree with probability 0.893, and the expected number of roots offering it is $10 \times 0.2 = 2$. In a bagged forest ($m_{\text{try}} = d$) it would be offered at all ten roots and, being the strongest feature, chosen at most of them.

(b) Explain why hiding x_1 from many roots may reduce the accuracy of individual trees but improve the forest.

Individual trees get weaker. A tree whose root cannot use x_1 must start with the best of five weaker features. Its first split removes less impurity, and because a tree is greedy and hierarchical [Chapter 5, §5.3], every later split lives inside the regions that first split created. Such a tree fits the training data less well and, on its own, predicts less accurately: its error variance σ^2 goes up a little. (x_1 is not lost: it is offered at 20% of all nodes, so the tree uses it deeper down.)

The forest gets stronger. The forest's error is the error of the *average* of the trees, and by the variance formula

$$\text{Var(forest)} = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).$$

Bagged trees that all start with x_1 have similar shapes and make correlated mistakes: ρ is large and the correlated part $\rho\sigma^2$ cannot be averaged away however many trees are added. Trees that start with different features partition the space differently, make mistakes in different places, and have a much smaller ρ. If, say, σ^2 rises by 10% while ρ falls from 0.6 to 0.3, the forest variance with $M = 100$ goes from $\sigma^2(0.6 + 0.004) = 0.604\sigma^2$ to $1.1\sigma^2(0.3 + 0.007) = 0.338\sigma^2$: a weaker tree, a much better forest. "Weaker trees, stronger forest" is just the statement that the correlation term dominates the formula once M is large.

(c) Explain how you would choose m_{try} and tree size for the whole forest. Why is it inappropriate to tune each tree separately?

Forest-level procedure.

1. Start at the usual default, $m_{\text{try}} \approx \sqrt{d} = 5$ for classification, and a tree-size setting (minimum leaf size, possibly maximum depth) shared by all trees.
2. For each candidate setting (m_{try}, leaf size) on a small grid, fit a *complete* forest and compute its OOB error [Chapter 5, §5.4], or its error on a separate validation set. OOB is free here: each observation is predicted by the roughly 37% of trees that did not see it, which is a genuine unseen-data estimate for the whole forest.
3. Pick the setting with the lowest OOB (or validation) error. Then increase the number of trees M until the OOB error stabilises; beyond the plateau more trees only cost time.
4. Keep a test set untouched for the final estimate: repeated OOB-based selection overfits the OOB estimate just as repeated validation overfits a validation set.

Why not tune each tree separately.

- *Wrong objective.* The quantity to minimise is the forest's error, not any tree's. Tuned on its own, every tree would choose what makes *it* most accurate: all features available ($m_{\text{try}} = d$) and a strong, deep fit. That reproduces the correlated bagged forest of part (b) and destroys the diversity that the forest needs. The best setting for a component is not the best setting for the average, precisely because of the ρ term.
- *The setting is a property of the ensemble.* m_{try} and tree size are ensemble hyperparameters ξ: $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$ with the same ξ for every m [Chapter 5, §5.4]. Changing ξ changes the forest's bias–variance balance; it is meaningful only at forest level.

- *Validity of the estimate.* The OOB error estimates the error of the *procedure* "fit M exchangeable trees with setting ξ and average". If each tree had its own tuned setting chosen by looking at its own OOB observations, the trees would no longer be exchangeable draws from one procedure and the OOB estimate would be contaminated by M separate selection steps.
- *Cost.* M separate searches instead of one, for a worse result.

3 Problem 3: Interpreting feature importance

Problem 3

For a node t, define its MDI credit as

$$\Delta_t = \frac{N_t}{N}\Big(I(S_t) - \frac{N_L}{N_t}I(S_L) - \frac{N_R}{N_t}I(S_R)\Big).$$

A root containing all N observations reduces Gini impurity from 0.50 to 0.45. A deeper node containing $0.30N$ observations reduces its impurity from 0.48 to 0.08.

What this tests

MDI credit is *weight* × *impurity reduction* [Chapter 5, §5.6]: the weight N_t/N is the fraction of data the split acts on and the bracket is the reduction from the parent to the size-weighted children. Permutation importance is the error increase after shuffling one feature in OOB or validation data, with no retraining [Chapter 5, §5.6]. Both measure what the *fitted model relies on*, which is neither the feature's intrinsic information (correlated features mask each other) nor causation.

(a) Compute the MDI credit of each split. Why can the deeper split receive more credit even though it contains fewer observations?

The statement "reduces impurity from a to b" gives the bracket directly: $I(S_t) = a$ and the size-weighted child impurity $\frac{N_L}{N_t}I(S_L) + \frac{N_R}{N_t}I(S_R) = b$ (this reading is the assumption).

$$\text{Root:} \quad \Delta = \frac{N}{N}(0.50 - 0.45) = 1 \times 0.05 = 0.05,$$

$$\text{Deeper node:} \quad \Delta = \frac{0.30N}{N}(0.48 - 0.08) = 0.30 \times 0.40 = 0.12.$$

The deeper split earns 0.12, more than twice the root's 0.05, although it acts on 30% of the data. The credit is a product: a *small* reduction (0.05) applied to *everyone* is worth $0.05N$ "units of impurity removed", while a *large* reduction (0.40) applied to 30% of the observations is worth $0.40 \times 0.30N = 0.12N$ units. Early splits have the larger weight factor but are often only mildly informative (they separate the data coarsely); a deeper split that nearly purifies its node ($0.48 \to 0.08$) can remove far more impurity in absolute terms. The weight N_t/N makes MDI count impurity removed per training observation, not per split.

(b) A fitted forest has validation error 0.12. After permuting x_a, the error is 0.20; after permuting x_b, it is 0.14. Compute the permutation importance of each feature. What do the two values say about how much the fitted forest relies on these features?

Permutation importance is the increase in error, $I_j = \widehat{\mathrm{Err}}_{\mathrm{shuffled}(j)} - \widehat{\mathrm{Err}}_{\mathrm{original}}$:

$$I_a = 0.20 - 0.12 = 0.08, \qquad I_b = 0.14 - 0.12 = 0.02.$$

Destroying the information in x_a costs the forest eight extra errors per hundred validation cases, two thirds more than its baseline error rate; destroying x_b costs two per hundred. The fitted forest relies on x_a about four times as heavily as on x_b (in the sense of error attributable to each). Three qualifications:

- these are statements about *this fitted forest's reliance in prediction*, measured honestly on data the trees did not see (validation here, OOB in the lecture); they are not statements about how much information the features "contain";
- 0.02 is small but not zero; whether it is distinguishable from noise depends on the validation set size, so repeat the shuffle several times and average, or report a spread;
- the baseline 0.12 matters: an increase of 0.08 on a baseline of 0.12 is dramatic, the same increase on a baseline of 0.40 would be modest.

(c) Suppose x_a and x_b are strongly correlated and carry similar information. Explain why shuffling only one of them may make it appear less important than it really is. Why does high feature importance not prove that the feature causes the outcome?

Masking by correlation. Permutation importance asks "how much worse does the forest predict when the information in x_j is destroyed?" If x_b carries nearly the same information as x_a, then after shuffling x_a the information is *still available* to the forest through x_b: trees that split on x_b are unaffected, and many trees that split on x_a also split on x_b nearby. The error rises only by the part of the information that is unique to x_a, not by the value of the shared information. Symmetrically for x_b. Each feature individually appears less important than the information it carries, and their two importances add up to less than the importance of the pair. (The asymmetry 0.08 versus 0.02 then tells you which of the two the forest happened to pick more often, which can be an accident of m_{try} draws and training-sample noise, not that x_a is "more informative".) Remedies: permute x_a and x_b *jointly* to measure the shared contribution; or drop one column, refit, and compare; or group correlated features before computing importance. MDI has the same masking problem for the same reason, plus its own bias towards features with many split points.

Importance is not causation. A high importance means the fitted model's predictions change a lot when the feature is altered. That is a fact about the model, not about the world. The feature may be

- a *proxy* for the true cause: destination port stands in for "which service is being attacked"; change the service's port number and the attacks are unchanged, but the forest's predictions are;
- a *consequence* of the outcome recorded after the fact (an alert flag, a response-time field), which is target leakage: it predicts because it was produced by the label, not the other way round;
- an artefact of the *data collection*: attacks captured on one day from one subnet make the subnet look causal;
- simply the member of a correlated group that the greedy splitter happened to choose.

Causation is a claim about what happens under *intervention*; importance is computed under *observation* of data generated by the existing mechanism. No amount of predictive reliance converts one into the other.

4 Problem 4: Fitting residuals with shrinkage

Problem 4

Consider four regression observations:

x	1	2	3	4
y	0	2	4	6

Use squared loss and initialize with $F_0 = \bar{y}$. A depth-one tree splits at $x = 2.5$.

What this tests

Gradient boosting for squared error [Chapter 5, §5.7]: initialise at the mean, compute residuals $r = y - F_{m-1}$, fit a tree to them with leaf outputs equal to the mean residual, update $F_m =$

$F_{m-1} + \eta T_m$. Shrinkage $\eta < 1$ applies only part of each correction and is the main control on overfitting together with the number of rounds and the tree depth.

(a) Compute the initial residuals and the mean residual in each leaf. Write the fitted correction tree $T_1(x)$.

Initialisation: $F_0 = \bar{y} = (0+2+4+6)/4 = 3$ for every x. Residuals $r_1^{(i)} = y^{(i)} - F_0$:

x	1	2	3	4
y	0	2	4	6
F_0	3	3	3	3
r_1	-3	-1	$+1$	$+3$

The split $x \le 2.5$ puts $x = 1, 2$ in the left leaf and $x = 3, 4$ in the right leaf. For squared loss the optimal leaf output is the mean residual in the leaf [Chapter 5, §5.3, 5.7]:

$$w_L = \frac{-3 + (-1)}{2} = -2, \qquad w_R = \frac{1+3}{2} = +2.$$

The fitted correction tree is

$$T_1(x) = \begin{cases} -2, & x \le 2.5, \\ +2, & x > 2.5. \end{cases}$$

(b) With learning rate $\eta = 0.5$, compute F_1 and the new residuals. Compare the squared-error sums before and after the update.

$F_1(x) = F_0 + \eta\, T_1(x) = 3 + 0.5\, T_1(x)$, so $F_1 = 2$ on the left leaf and 4 on the right leaf.

x	1	2	3	4	sum of squares
$r_1 = y - F_0$	-3	-1	1	3	$9+1+1+9 = \mathbf{20}$
F_1	2	2	4	4	
$r_2 = y - F_1$	-2	0	0	2	$4+0+0+4 = \mathbf{8}$

The squared-error sum falls from 20 to 8 (the loss $L = \frac{1}{2}\sum r^2$ of the lecture falls from 10 to 4). For comparison, the full step $\eta = 1$ would give $F_1 = (1, 1, 5, 5)$, residuals $(-1, 1, -1, 1)$ and a sum of squares of 4: the unshrunk tree removes more training error in this round, as it must, since the mean residual is the least-squares optimum for these leaves. With $\eta = 0.5$ the ensemble has taken half of that step.

Do not confuse

The learning rate is applied in the *update*, not in the fit: T_1 is fitted to the full residuals and then scaled, $F_1 = F_0 + \eta T_1$. Fitting the tree to ηr instead would give the same thing here, but the convention matters when the leaf value is a Newton step (Problem 5).

(c) When $\eta < 1$, the first tree corrects only part of the residuals. Explain why leaving some error for later trees can improve validation performance.

Several mechanisms, all visible in this tiny example.

- *Each tree commits less.* A depth-one tree can only express one step; the full step ($\eta = 1$) immediately commits the ensemble to $F = 1$ on the left and 5 on the right. With $\eta = 0.5$ the remaining residuals $(-2, 0, 0, 2)$ still carry the *structure* the first tree could not express, the slope inside each leaf. The next tree, fitted to those residuals, may split at $x = 1.5$ or $x = 3.5$ and start recovering the linear trend; with $\eta = 1$ the leftover $(-1, 1, -1, 1)$ looks like alternating noise and a second depth-one tree can do little with it. Shrinkage lets later trees re-partition the space instead of merely patching the first tree's regions [Chapter 5, §5.7, "why many shallow trees"].

- *Averaging over many partial fits.* The final model is $F_0 + \eta \sum_m T_m$: a sum of many small corrections, each fitted to slightly different residuals, rather than a few large ones. Sample-specific noise in the residuals is fitted only a fraction η at a time and has many rounds in which to be averaged out by the other trees; the resulting function is smoother and less dependent on any one tree's split choice. This is variance reduction in the sense of Chapter 3.
- *Finer control of capacity.* The number of rounds M is chosen by early stopping on validation [Chapter 5, §5.7]. With a small η, successive models differ little, so the validation curve is smooth and the stopping point is found reliably; with $\eta = 1$ the capacity jumps in large steps and the minimum can be overshot.

The price is computation: smaller η needs more rounds ($\eta \propto 1/M$). The gain is a better validation loss, provided M is tuned together with η and the depth, not in isolation.

5 Problem 5: From pseudo-residuals to Newton leaf values

Problem 5

For binary cross-entropy, let $F^{(i)}$ be the current logit and $p^{(i)} = \sigma(F^{(i)})$.

What this tests

Boosting for classification works in logit space [Chapter 5, §5.8]. The gradient and Hessian of the cross-entropy with respect to the logit are $g = p - y$ and $h = p(1-p)$; a second-order expansion of the loss in a leaf gives the closed-form Newton leaf value $\sum(-g)/(\sum h + \lambda)$, which replaces the numerical line search. The ratio $-g/h$ is XGBoost's per-observation target and h its weight; they must be read together [Chapter 5, §5.9].

(a) Starting from $\ell(y, F) = -y \log p - (1-y)\log(1-p)$, prove that $g^{(i)} = \dfrac{\partial \ell^{(i)}}{\partial F^{(i)}} = p^{(i)} - y^{(i)}$ and $h^{(i)} = \dfrac{\partial^2 \ell^{(i)}}{\partial (F^{(i)})^2} = p^{(i)}(1-p^{(i)})$.

Drop the index i. *Step 1: derivative of the sigmoid.* With $p = \sigma(F) = (1+e^{-F})^{-1}$,

$$\frac{dp}{dF} = \frac{e^{-F}}{(1+e^{-F})^2} = \frac{1}{1+e^{-F}} \cdot \frac{e^{-F}}{1+e^{-F}} = p\,(1-p),$$

because $\frac{e^{-F}}{1+e^{-F}} = 1 - \frac{1}{1+e^{-F}} = 1 - p$.

Step 2: derivative of the loss in p.

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}.$$

Step 3: chain rule.

$$g = \frac{\partial \ell}{\partial F} = \frac{\partial \ell}{\partial p}\frac{dp}{dF} = \Big(-\frac{y}{p} + \frac{1-y}{1-p}\Big)p(1-p) = -y(1-p) + (1-y)p = -y + yp + p - yp = p - y.$$

Step 4: second derivative. $g = p - y$ depends on F only through p, and y is a constant, so

$$h = \frac{\partial g}{\partial F} = \frac{dp}{dF} = p(1-p). \qquad \blacksquare$$

Interpretation: $-g = y - p$ is the classification analogue of the residual (how far the probability is from the label); $h = p(1-p)$ is the Bernoulli variance, maximal (0.25) when the model is unsure and vanishing when it is confident. Note $h > 0$ always, so the quadratic approximation below is convex.

(b) A proposed leaf contains three observations: $(y^{(i)}, p^{(i)}) = (1, 0.2), (1, 0.4), (0, 0.8)$. Compute $G = \sum_i g^{(i)}$, $H = \sum_i h^{(i)}$, and the Newton leaf value both without regularization and with $\lambda = 1$. If $\eta = 0.1$, what correction is actually added to the logit?

y	p	$g = p - y$	$h = p(1-p)$	$-g/h$
1	0.2	-0.8	0.16	5.00
1	0.4	-0.6	0.24	2.50
0	0.8	$+0.8$	0.16	-5.00
		$G = -0.6$	$H = 0.56$	

The leaf's second-order objective is $\phi(w) = Gw + \frac{1}{2}(H + \lambda)w^2$, minimised at $w^* = -G/(H + \lambda)$ [Chapter 5, §5.8, 5.9].

$$\text{Without regularisation } (\lambda = 0): \quad w^* = \frac{-G}{H} = \frac{0.6}{0.56} = 1.071,$$

$$\text{With } \lambda = 1: \quad w^* = \frac{-G}{H + 1} = \frac{0.6}{1.56} = 0.3846.$$

The correction actually added to the logit of every observation in the leaf is $\eta\, w^*$:

$$\eta w^* = 0.1 \times 0.3846 = 0.0385 \quad \text{(regularised)}, \qquad 0.1 \times 1.071 = 0.107 \quad \text{(unregularised)}.$$

Sanity check of the sign: two of the three observations are positives whose probabilities are too low; the leaf value is positive and raises all three logits. After the regularised step the probabilities become $\sigma(\text{logit}(0.2) + 0.0385) = 0.206$, $\sigma(\text{logit}(0.4) + 0.0385) = 0.409$ and $\sigma(\text{logit}(0.8) + 0.0385) = 0.806$: a small improvement for the two positives at a small cost for the negative, which is what a shrunk step should do. Notice also that the leaf value is *not* the average of the per-observation targets $-g/h$ (which would be $(5 + 2.5 - 5)/3 = 0.83$); it is their h-weighted average, $\sum h\,(-g/h)/\sum h = \sum(-g)/\sum h$, which is the point of part (c).

(c) For one observation, let p be very close to zero. Its Newton pseudo-target $-g/h$ can then be very large. What happens to its weight h at the same time, and why must the target and weight be considered together?

Take $y = 1$ and $p \to 0$ (a positive the model is confidently wrong about). Then

$$-g = y - p = 1 - p \to 1, \qquad h = p(1-p) \to 0, \qquad r_{\text{XGB}} = \frac{-g}{h} = \frac{1-p}{p(1-p)} = \frac{1}{p} \to \infty.$$

The target explodes because the second-order model of the loss says: "the loss is almost flat in F here ($h \approx 0$), so to remove a residual of size 1 you need an enormous step". But the *weight* of that observation in the leaf objective is the same $h \to 0$: in XGBoost's weighted least-squares form $\sum_i \frac{1}{2} h^{(i)} (r^{(i)}_{\text{XGB}} - w)^2$ [Chapter 5, §5.9], this observation's term is multiplied by a vanishing weight, so its enormous request is almost ignored. What actually enters the leaf value $w^* = \sum(-g)/(\sum h + \lambda)$ is the *product* $h \cdot r_{\text{XGB}} = -g = 1 - p$, which is bounded by 1: the observation contributes at most 1 to the numerator and almost nothing to the denominator. (If instead $y = 0$ and $p \to 0$, the model is confidently *right*: $-g = -p \to 0$, $r_{\text{XGB}} = -1/(1-p) \to -1$, nothing explodes.)

Reading the target alone would suggest a leaf value in the thousands; reading the weight alone would suggest the observation is irrelevant. Both are wrong: the observation pulls the leaf *in the right direction* (towards positive logits) with a *bounded* force. This is why XGBoost never fits the raw targets r_{XGB} by unweighted least squares, which would let a single confidently-wrong point dictate a leaf.

There is one genuine danger left: a leaf made *only* of such observations has $\sum h \approx 0$, and $w^* = \sum(-g)/\sum h$ does explode. That is exactly what λ in the denominator prevents ($w^* \leq \sum(-g)/\lambda$), and what the *minimum child weight* control enforces by refusing to create a leaf whose $\sum h$ is too small [Chapter 5, §5.9].

6 Problem 6: A split must pay for its extra leaf

Problem 6

An XGBoost parent node has $(G, H) = (-4, 3)$. A candidate split produces left child $(G_L, H_L) = (-3, 1)$ and right child $(G_R, H_R) = (-1, 2)$. Let $\lambda = 1$.

What this tests

XGBoost scores a tree with one regularised second-order objective [Chapter 5, §5.9]. Each leaf contributes $\phi(w) = Gw + \frac{1}{2}(H+\lambda)w^2$, minimised at $w^* = -G/(H+\lambda)$ with value $-\frac{1}{2}G^2/(H+\lambda)$. A split is accepted when the children's total minimum beats the parent's by more than the per-leaf price γ: gain $= \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma > 0$.

(a) For a leaf with derivative sums (G, H), its regularized quadratic contribution is $\phi(w) = Gw + \frac{1}{2}(H+\lambda)w^2$. Find the value w^* that minimizes $\phi(w)$, then substitute it into ϕ to obtain the minimum $\phi(w^*)$. Use these minimum values to calculate how much the objective decreases when the parent is replaced by the two children. After subtracting $\gamma = 0.2$, should XGBoost accept the split?

Minimiser. ϕ is a convex quadratic ($H + \lambda > 0$). Setting $\phi'(w) = G + (H+\lambda)w = 0$,

$$w^* = -\frac{G}{H+\lambda}.$$

Minimum value.

$$\phi(w^*) = G\Big(-\frac{G}{H+\lambda}\Big) + \frac{1}{2}(H+\lambda)\frac{G^2}{(H+\lambda)^2} = -\frac{G^2}{H+\lambda} + \frac{1}{2}\frac{G^2}{H+\lambda} = -\frac{1}{2}\frac{G^2}{H+\lambda}.$$

Apply to the three nodes with $\lambda = 1$.

Node	(G, H)	$H+\lambda$	$w^* = -G/(H+\lambda)$	$\phi(w^*) = -\frac{1}{2}G^2/(H+\lambda)$
parent	$(-4, 3)$	4	1.000	$-\frac{1}{2}\cdot 16/4 = -2.000$
left	$(-3, 1)$	2	1.500	$-\frac{1}{2}\cdot 9/2 = -2.250$
right	$(-1, 2)$	3	0.333	$-\frac{1}{2}\cdot 1/3 = -0.167$

(Check: $G_L + G_R = -4 = G$ and $H_L + H_R = 3 = H$, as they must.)

Decrease in objective. Replacing the parent leaf by the two children changes the data-fit part from -2.000 to $-2.250 - 0.167 = -2.417$, a decrease of

$$(-2.000) - (-2.417) = 0.417 = \frac{5}{12}.$$

Equivalently, $\frac{1}{2}\left[\frac{9}{2} + \frac{1}{3} - \frac{16}{4}\right] = \frac{1}{2}[4.5 + 0.333 - 4] = \frac{1}{2}(0.833) = 0.417$.

Subtract the leaf price. gain $= 0.417 - 0.2 = 0.217 > 0$. **Accept the split.** The two children, each with its own output (1.5 and 0.33 instead of a single 1.0), fit the gradients better than the parent by 0.417, which more than pays the 0.2 charged for the additional leaf.

(b) Find the largest value of γ for which the split is accepted.

The split is accepted while gain $= 0.417 - \gamma > 0$, i.e. for $\gamma < \frac{5}{12} \approx 0.417$. At $\gamma = 5/12$ exactly the gain is zero; under the rule "accept only when the gain is strictly positive" [Chapter 5, §5.9] the split is then rejected, so $5/12$ is the supremum of accepted values rather than itself accepted (an implementation that accepts a zero gain would include it). In words: the improvement in fit, 0.417, is the most this split can afford to pay for its extra leaf.

(c) A binary split replaces one leaf with two. Use the penalty γQ to explain why this costs exactly one γ. How does this effect differ from increasing λ?

Why exactly one γ. The complexity term in the XGBoost objective is γQ_m, with Q_m the number of leaves of the tree. A binary split removes one leaf (the parent) and creates two (the children), so $Q_m \to Q_m + 1$ and the penalty changes by

$$\gamma(Q_m + 1) - \gamma Q_m = \gamma.$$

It is independent of which node is split, of how many observations it holds, and of G, H or λ: a flat admission fee per leaf. That is why γ appears as a plain subtraction in the gain formula and acts as a *threshold*: splits whose fit improvement is below γ are not made (pre-pruning), whatever else is true of them.

How λ differs. λ enters through the term $\frac{\lambda}{2}\sum_q w_q^2$ and therefore sits *inside* every leaf's score, $-\frac{1}{2}G_q^2/(H_q+\lambda)$, and inside every leaf's output, $w_q^* = -G_q/(H_q+\lambda)$.

- It is *continuous shrinkage*, not a fixed fee: every leaf output is pulled towards zero, and every one of the three terms in the gain is reduced.
- Its effect is *uneven*: it matters most where H_q is small (few observations, or confident ones with $h \approx 0$). In the example, adding $\lambda = 1$ to the right child ($H = 2$) changes its score from -0.25 to -0.167, while for a leaf with $H = 50$ it would barely register. λ therefore specifically disfavours splits that create small or low-curvature leaves, the ones whose Newton steps would explode (Problem 5c).
- It also reduces gains indirectly: the gain of this split without λ would be $\frac{1}{2}[9/1+1/2-16/3] = 2.083$, and with $\lambda = 1$ it is 0.417. So raising λ does make splits rarer, but by shrinking the children's *apparent* improvement in proportion to their size, not by charging a constant.

In one line: γ charges a fixed price per leaf and decides *whether* to split; λ shrinks leaf outputs and decides *how far* each leaf moves, biting hardest on small leaves.

7 Problem 7: Choosing a model from validation results

Problem 7

Five gradient-boosting configurations produce the following results. The selected round is the round with minimum validation log loss.

Configuration	η	depth	selected round	training loss	validation loss	fit time (s)
A	0.10	1	80	0.31	0.34	12
B	0.10	3	110	0.12	0.29	28
C	0.10	5	70	0.05	0.35	42
D	0.03	3	340	0.14	0.27	75
E	0.03	5	240	0.04	0.30	110

What this tests

Model selection uses validation loss, never training loss (Chapter 3). The boosting triad M, η, depth is coupled: smaller η needs more rounds, deeper trees overfit sooner [Chapter 5, §5.7]. Early stopping selects M per configuration. The test set is touched once, after every decision is frozen.

(a) If the selection criterion is validation log loss, which configuration should be chosen? Why would choosing by training loss give a different answer?

Choose D ($\eta = 0.03$, depth 3, 340 rounds): validation log loss 0.27, the lowest of the five. Ranking by validation loss: D (0.27) < B (0.29) < E (0.30) < A (0.34) < C (0.35).

Choosing by training loss would pick **E** (0.04), with C (0.05) second: the ranking by training loss is almost the reverse of the ranking by validation loss at the top. Training loss rewards *capacity*: deeper trees (depth 5) can fit the training observations, including their noise, much more closely, so their training loss is tiny. But that fit does not transfer to new data; the depth-5 models have the two *largest* gaps between training and validation loss (0.30 for C, 0.26 for E) and the worse validation losses. Training loss is an optimistically biased estimate of generalisation whose bias grows with capacity, so it systematically favours the most complex configuration (Chapter 3).

(b) Which configurations appear too complex for these data? What plot would provide stronger evidence of overfitting?

C and E, the depth-5 configurations. The signature of excess capacity is a very low training loss together with a validation loss that is no better, or worse, than that of a simpler model:

	depth	training	validation	gap
A	1	0.31	0.34	0.03
B	3	0.12	0.29	0.17
C	5	0.05	0.35	0.30
D	3	0.14	0.27	0.13
E	5	0.04	0.30	0.26

At each learning rate the depth-5 model has lower training loss but higher validation loss than the depth-3 model (C vs B: 0.35 vs 0.29; E vs D: 0.30 vs 0.27). Depth 5 allows fifth-order feature interactions that these data do not support. Note also that C stops earliest (70 rounds): its validation curve turns up quickly, another sign that each round adds too much capacity. At the other end A (depth 1, additive effects only) looks slightly *underfit*: a small gap but a high loss on both sets.

Stronger evidence: learning curves. Plot training loss and validation loss against the boosting round for each configuration (the lecture's "boosting and the risk of overfitting" figure, [Chapter 5, §5.7]). Overfitting shows as the validation curve reaching a minimum and *rising* while the training curve keeps falling; the depth of the rise after the minimum, and how early it occurs, quantify the excess capacity far better than one number per configuration. For C and E we would expect a sharp early minimum followed by a steady climb; for D a shallow, late, flat minimum. A second useful plot is validation loss against depth at each η, which should show a U shape with its minimum at depth 3.

(c) If training must finish within 40 seconds, which configuration would you recommend? State the loss in predictive performance caused by this time limit.

Only A (12 s) and B (28 s) meet the limit; C (42 s) just misses it, and D, E are far over. **Recommend B**: validation log loss 0.29. The cost of the time constraint is the difference from the unconstrained best, D: $0.29 - 0.27 = 0.02$ in validation log loss (about 7% relative). Two remarks: the 40 s budget forces the larger learning rate (0.10), which is the whole reason D is unavailable, since D is just B with η reduced and M tripled; and whether 0.02 is a meaningful loss depends on the size of the validation set and on what the log loss translates into for the actual decision (Chapter 2), which this table does not show.

(d) Explain why the smaller learning rate generally selects more rounds. Does this table prove that a smaller learning rate is always better?

Why more rounds. Each round adds ηT_m to the ensemble. With a smaller η every tree contributes a smaller fraction of its fitted correction, so the ensemble moves towards the training targets in smaller steps and needs more steps to reach the same degree of fit; roughly, the total "amount of correction" ηM is what matters, so $\eta \propto 1/M$ [Chapter 5, §5.7]. The table shows this directly: B ($\eta = 0.10$) stops at 110 rounds and D ($\eta = 0.03$) at 340, a ratio of 3.1 for a learning-rate ratio of 3.3; C (70) and E (240) give 3.4. The minimum of the validation curve simply occurs later when the steps are smaller.

Does the table prove smaller η is always better? No.

- *Two comparisons only.* D beats B by 0.02 and E beats C by 0.05. That is consistent with the general tendency (smaller steps give a smoother additive fit and a more reliably located minimum), but it is two paired observations at one depth each, on one validation set, with no uncertainty attached. A difference of 0.02 in log loss can be within validation noise.
- *Selection bias.* The validation set was used to pick the stopping round of all five configurations; the reported validation losses are therefore slightly optimistic, and not necessarily equally so.
- *Confounded with cost.* Smaller η bought the improvement with roughly three times the fit time. "Better" depends on whether that time is available (part c).
- *Diminishing returns.* The tendency is that reducing η helps until the validation curve is smooth, then stops helping; nothing in the table says $\eta = 0.01$ would beat 0.03.

The honest conclusion is "at both depths, the smaller learning rate gave a lower validation loss at a higher cost in these data"; a general claim would need repeated splits or cross-validation and more values of η.

> **(e)** After choosing the configuration, explain how to use the training, validation, and test sets without allowing the test results to influence any modeling decision.

The three sets have three jobs that must never be swapped (Chapter 3).

1. **Training set: fit.** Every tree, every leaf value, is computed from the training set alone.
2. **Validation set: choose.** Every decision that is not a parameter is made here: η, depth, the stopping round, λ, γ, subsampling, and any comparison between configurations. This is what the table did. Because the validation set has been used for selection, its loss for the chosen model is no longer an unbiased estimate of generalisation.
3. **Test set: report, once.** After D with 340 rounds has been fixed, and *only* then, predict the test set once and report the loss (and the decision metrics that matter). The number is reported as is. If it disappoints, the correct response is *not* to go back and try C, or to change the stopping round: that would turn the test set into a second validation set and make the final number optimistic in a way that cannot be measured. Any further iteration requires fresh test data or a re-split with the old test data folded into development.

Optional refinement, done *before* touching the test set: refit the chosen configuration on training plus validation data with the hyperparameters and round count frozen (Problem 8c discusses when this is reasonable), then evaluate that refit model once on the test set. Keep a written protocol of what was decided when, so that "the test set influenced nothing" is a checkable statement.

8 Problem 8: Avoiding leakage in an ensemble experiment

Problem 8

A team compares a decision tree, random forest, gradient boosting model, and XGBoost. After viewing the test results, it repeatedly changes the hyperparameters. For stacking, it trains the meta-model using predictions that the base models made on their own training observations.

What this tests

Two forms of leakage [Chapter 5, §5.4, 5.12]: using the test set for selection (the test set becomes a validation set, and its estimate becomes optimistic), and training a stacking meta-model on in-sample base predictions (the meta-model learns to trust whichever base model memorised the training data best). OOB predictions are a bootstrap by-product inside a bagged ensemble; out-of-fold predictions come from an explicit fold split and work for any model.

(a) Identify the two ways in which this procedure gives results that are too optimistic. Then describe a correct training, model-selection, and test procedure.

Leak 1: the test set is used for model selection. Changing hyperparameters after looking at test results and keeping the setting that scored best is selection *on* the test set. Over many tries the best test score drifts upward partly by luck (the "winner's curse" of repeated comparison, Chapter 3); the final reported test loss is then an optimistic estimate of performance on genuinely new data, and there is no remaining data on which to measure how optimistic. The test set has silently become a validation set.

Leak 2: the stacking meta-model is trained on in-sample base predictions. A deep decision tree, a random forest and a well-fitted boosting model predict their *own* training observations almost perfectly (a tree with pure leaves reproduces its training labels exactly). The meta-model therefore sees base predictions that look far more accurate than they will be on new data, learns to put large weight on the models that overfit most, and its own fit to the training labels is unrealistically good. At prediction time the base models are far less accurate on new inputs than the meta-model was trained to assume. Both the stacked model's quality and its estimated quality are inflated [Chapter 5, §5.12].

A correct procedure.

1. *Split once*, stratified: training, validation and test (or development plus test, with cross-validation inside development). The same rows for every model, as Lab 3 requires.
2. *Fit* every candidate (tree, forest, GBM, XGBoost) on the training set only.
3. *Select* all hyperparameters on the validation set or by K-fold cross-validation within the training set: tree depth, m_{try} and leaf size for the forest (OOB error is also legitimate here), η, depth and rounds by early stopping for the boosters, λ and γ for XGBoost. Compare the tuned models on the validation data.
4. *Stack honestly*: build the meta-model's training inputs from *out-of-fold* predictions (part b), fit a simple regularised meta-model, and treat the stacked model as one more candidate evaluated on validation data.
5. *Freeze* every choice. Optionally refit the winners on training plus validation (part c).
6. *Test once*: predict the test set with the frozen models and report. Do not iterate.

(b) What is the difference between OOB predictions and out-of-fold predictions? Describe how to produce the out-of-fold predictions needed to train the stacking meta-model.

OOB predictions [Chapter 5, §5.4] exist only inside a *bagged* ensemble. Each tree is fitted on a bootstrap sample that omits about 36.8% of the training observations; the OOB prediction for observation i averages the roughly $0.368M$ trees that did not see i. It costs nothing extra, it uses the *same* trees that form the final forest (each observation is predicted by a subset of them), and it is specific to the bootstrap mechanism: a single decision tree, a boosting model or a logistic regression has no OOB predictions.

Out-of-fold predictions come from an explicit K-fold partition of the training set. For each fold k, a *separate* copy of the model is fitted on the other $K-1$ folds and predicts fold k. Every observation receives exactly one prediction, from a model that was fitted without it, using $(K-1)/K$ of the data. It requires K extra fits per base model, applies to *any* model type, and the models used to produce them are not the final models (the final models are refitted on all training data).

Both deliver "a prediction for a training point from a model that never saw it"; OOB is the bootstrap by-product, out-of-fold is the general-purpose construction. Stacking needs out-of-fold predictions because its base models include non-bagged ones.

Producing them for the stacking meta-model.

1. Partition the training set into K folds (say $K = 5$), stratified by class; use the *same* folds for every base model.

2. For each base model m (tree, forest, GBM, XGBoost) and each fold k: fit m on folds $\neq k$ with its already-selected hyperparameters, and predict $\hat{p}_m(y = 1 \mid x^{(i)})$ for every i in fold k. Store it as $z_m^{(i)}$.
3. After all folds, each training observation i has a vector $z^{(i)} = (z_1^{(i)}, \ldots, z_M^{(i)})$ of honest base predictions. Assemble the $N \times M$ matrix Z.
4. Fit the meta-model, e.g. a regularised logistic regression $\hat{p}_{\text{stack}} = \sigma(\beta_0 + \sum_m \beta_m z_m)$, on (Z, y) [Chapter 5, §5.12].
5. For *prediction*, refit each base model on the entire training set, compute $(z_1, \ldots, z_M)$ for the new input, and pass them through the fitted meta-model. (Training and prediction use different base-model fits; that is intended.)

If hyperparameters of the base models must also be tuned, do it inside each fold (nested cross-validation) or on a separate validation set, so that the out-of-fold predictions remain predictions from models that never saw the observation in any way.

(c) Once every modeling choice has been fixed, the team considers refitting the base models using both the training and validation data. When is this reasonable, and how can the test set still provide a fair final evaluation?

When it is reasonable. Refitting on training plus validation is reasonable when (i) every modelling choice is genuinely frozen: model family, all hyperparameters, the number of boosting rounds, the meta-model and its inputs; no decision will be revisited after the refit; (ii) the hyperparameters are expected to *transfer* to a somewhat larger sample. Settings like depth, m_{try}, η, λ transfer well; the number of boosting rounds selected by early stopping is tied to the training-set size and the chosen value may be slightly too small for a 25% larger set, so keep it fixed (safe) or scale it modestly by a rule decided in advance, not by looking at results; (iii) more data matters: with a small training set the gain from 25% more observations is real, with a very large one it is negligible and the refit only adds risk. It is also reasonable only if the stacking meta-model is handled consistently: either refit the base models on training plus validation and regenerate out-of-fold predictions on that larger set before refitting the meta-model, or keep the meta-model as fitted; mixing a meta-model fitted on old base predictions with newly refitted base models silently changes what its inputs mean.

Why the test set still gives a fair evaluation. The test set was not used for *any* choice: not for selecting hyperparameters, not for deciding to refit, not for the refit itself. The refit model is one fixed function, decided before the test set is opened. Evaluating a fixed function once on data it has never influenced is an unbiased estimate of that function's performance on the distribution the test set represents. What is lost is the ability to *validate the refit model itself*: there is no held-out data left on which to check that the refit behaved as expected before the test. The team therefore trusts the refit on the strength of the procedure, which is why the hyperparameters must be frozen and why any surprise on the test set must be reported, not fixed.

9 Problem 9: Deployment when the data may change

Problem 9

A security team finds that its deep decision tree changes substantially when the training sample changes. A random forest is more stable, but it relies heavily on destination port. The validation data contain the same services as the training data, while the deployed system will also encounter new services and ports.

What this tests

Bagging and forests cure *instability* (variance); boosting cures *structured residual error* (bias); neither cures *missing information* or *distribution shift* [Chapter 5, §5.11]. Validation on data that

share the training distribution cannot certify performance on inputs outside it (Chapter 3). A grouped or temporal split that holds out whole services is the evaluation design that matches the deployment question.

(a) Explain what bagging, a random forest, and boosting could each help with in this situation. Design an experiment that separates improvement due to the ensemble from improvement due to using destination port.

What each method addresses.

- *Bagging.* The symptom "the deep tree changes substantially when the training sample changes" is high variance. Bagging averages many deep trees fitted to bootstrap samples; by $\text{Var} = \sigma^2(\rho + (1-\rho)/M)$ [Chapter 5, §5.2] the average fluctuates far less than one tree while keeping the trees' ability to model interactions. It directly treats the instability. It does not change what information the trees use: if every tree splits on port at the root, bagging inherits the reliance.
- *Random forest.* In addition to averaging, m_{try} hides destination port from about $1 - m_{\text{try}}/d$ of the nodes [Chapter 5, §5.5]. Trees are forced to find alternative, behavioural splits (duration, bytes, packet timing), which decorrelates them and, as a side effect, *reduces* the forest's dependence on port, though it does not remove it, since port is still offered at m_{try}/d of the nodes and is the strongest split when offered. The permutation importance of port will tell how much reliance remains.
- *Boosting.* Boosting attacks *structured* residual error: attack types or regions of feature space that the current model systematically gets wrong [Chapter 5, §5.7]. If the forest's errors are concentrated (e.g. one attack class is always missed), gradient boosting with shallow trees can correct them round by round. It does nothing for the port problem, and it is more sensitive to label noise and to concentrating on hard cases, so it needs early stopping on validation.
- *None of them* supplies information that is absent, and none of them can learn how *unseen* ports behave [Chapter 5, §5.11]: a new port value falls into whatever leaf its threshold comparisons send it to, arbitrarily.

An experiment that separates the two effects. A two-factor design with identical data for every cell:

Model (factor 1)	with destination port	without destination port
single deep tree	cell 1	cell 2
bagged trees	cell 3	cell 4
random forest	cell 5	cell 6
gradient boosting / XGBoost	cell 7	cell 8

1. Use one fixed stratified split and the *same rows* in every cell (Lab 3's rule); tune each model's hyperparameters on the validation set within its own column, so the comparison is between tuned models.
2. Report the validation metrics that matter for the alerting decision (recall per attack class, false-alarm rate at the operating point, log loss), not only accuracy.
3. *Ensemble effect* = difference between rows within a column (e.g. cell 5 minus cell 1 with port; cell 6 minus cell 2 without). *Port effect* = difference between columns within a row (cell 1 minus cell 2, cell 5 minus cell 6). The *interaction*, whether the ensemble's gain is smaller without port, tells whether the ensemble was mostly exploiting port.
4. Measure *stability* directly, since that was the complaint: repeat each cell over several bootstrap resamples or random seeds of the training set and report the spread of the metrics and, for trees, how often the root split changes. Bagging and the forest should shrink this spread; the single tree should not.
5. Compute permutation importance of destination port in every cell of the "with port" column, to quantify remaining reliance.

(b) What validation results would make you trust the chosen model on the known services? Why would these results still not establish performance on new services?

Evidence for trust on known services.

- Low validation error (and good decision metrics at the chosen operating point) that is *stable* across repeated splits and seeds, and consistent between OOB and held-out validation for the forest: the number is not an accident of one split.
- Good performance *per service and per attack class*, not only on aggregate: a model that is excellent on HTTP and useless on DNS has a hidden failure.
- Reasonable calibration of the probabilities, so that the threshold chosen on validation means what it says in deployment.
- A learning curve that has flattened (more data would not change the model much) and a small training–validation gap.
- Permutation importance spread over several behavioural features rather than concentrated on port alone, and an ablation (part a's "without port" column) showing that the model is still useful when port is removed.
- Agreement between different model families on which cases are hard, suggesting the remaining errors are in the data, not in the model.

Why this says nothing about new services. Every validation observation comes from the same services and ports as the training data: the validation set is a sample from the *training* distribution. Generalisation guarantees (Chapter 3) are statements about new samples from the *same* distribution; they are silent about inputs from a different one. For a new port value, the model has never seen a single example with that value, and nothing in the validation error measured its behaviour there. Concretely, the trees compare the new port number with learnt thresholds and route the flow to a leaf that was shaped by *other* ports; whether that leaf's label is right for the new service is a matter of luck. The forest's stability on known services (low variance) is not accuracy on unknown ones (which is a bias problem under shift). "Test data differ substantially from the training data" is one of the listed situations where ensembles do not help [Chapter 5, §5.11], and validation on in-distribution data cannot detect it.

(c) The team proposes stacking all the models to handle the new services. Explain why stacking may not solve this problem. Suggest a change to the data split or evaluation design that would provide more useful evidence.

Why stacking does not address it.

- *Shared missing information.* Stacking combines base models that were all trained on the same services, all learnt the same port-to-service associations, and all lack any example of the new services. Their errors on new services will be *correlated* (they all extrapolate from the same wrong place), and a combination of correlated errors does not cancel them [Chapter 5, §5.1, 5.2]: "all models are missing the same important information" is the first item on the list of when ensembles do not help [Chapter 5, §5.11].
- *The meta-model is in-distribution too.* It is fitted on out-of-fold predictions for *known* services, so it learns how much to trust each base model *on known services*. It has no information about which base model degrades gracefully on new ports; it may well learn to trust the port-heavy model most, because that model is the best on the known data.
- *The symptom is bias under shift, not variance.* Stacking, like every fixed combination, is a variance-reduction and complementarity device. The problem here is that the deployment distribution has support (new port values) that the training distribution lacks. No reweighting of models trained on the old support fixes that.

A more useful evaluation design.

- *Grouped split by service.* Hold out *entire services* (all flows on a set of ports) from training and use them as the validation set: leave-one-service-out or grouped K-fold with service as the group.

The validation error then measures exactly the question "how does the model do on a service it has never seen?" Compare models, and the with/without-port columns of part (a), under this split. Expect the port-reliant models to collapse and the behavioural models to hold up; that difference is the evidence the team needs.

- *Temporal split.* If new services appear over time, train on an earlier period and validate on a later one in which new services have appeared; this also captures drift in attack behaviour.
- *Representation change.* Replace the raw port number by features that transfer, service category, whether the port is a registered service, behavioural statistics, so that a new port is described by properties the model has seen; then re-evaluate under the grouped split.
- *Reject option and monitoring.* Add a novelty check (an input far from all training services triggers review rather than a confident label, Chapter 4), and monitor the distribution of ports and of predicted probabilities in deployment to detect when the model is being asked a question its validation never covered.

Stacking may still be useful on known services; the honest statement is that its value on new services can only be demonstrated by an evaluation that holds new services out.

The thread so far

The nine problems, in one breath. Combine models when their errors differ (1); make them differ by randomising rows and features, and tune the ensemble, not its members (2); read importance as reliance of the fitted model, masked by correlation, never as causation (3); boosting adds shrunken corrections to residuals (4), to logits via Newton steps whose target and weight must be read together (5), and accepts a split only when the fit improvement pays the fixed price γ of a leaf while λ shrinks outputs (6); choose by validation, stop early, test once (7); avoid both selection on the test set and in-sample stacking inputs, using out-of-fold predictions (8); and remember that no ensemble supplies missing information or survives a shift that the evaluation design never measured (9).